Stress-Adaptive Action Governance for Autonomous AI Agents

Anonymous Authors

Abstract

Current approaches to AI agent safety rely primarily on static guardrails — fixed allow/deny lists that either block forbidden actions or permit everything else, regardless of situational context. We present a stress-adaptive governance model that dynamically restricts an agent's action vocabulary in response to environmental pressure. The model defines four stress levels (LOW, MED, HIGH, OVER) that progressively tighten behavioral constraints through a deterministic pipeline: observations activate behavioral modes via exponential moving averages, mode activation drives stress classification, and stress level determines the set of actions available to the agent at each decision cycle. Under extreme stress (OVER), the agent's action space collapses to a minimal safe set of four reactive and disengagement actions. We evaluate this model against a static action filter baseline across four scenarios totaling 1,200 decision cycles. In the stress-spike scenario, the adaptive model escalates through all four levels, restricts the average action space from 12 to ~6 actions, and spends 58 turns in HIGH or OVER states — while the static baseline remains permanently at LOW with all 12 actions available. Both systems achieve identical forbidden-action blocking (100%), demonstrating that the adaptive model's advantage lies not in what it blocks, but in how it governs what remains permitted. We argue that stress-adaptive governance addresses a critical gap between static guardrails and unconstrained agent autonomy.

1. Introduction

The deployment of autonomous AI agents — systems that take actions in external environments based on language model reasoning — has created an urgent need for runtime behavioral governance. Current safety approaches fall into two broad categories. The first is prompt-level guardrails: system instructions, constitutional principles, or RLHF training that attempt to shape the model's behavior before deployment. These approaches are well-studied but fundamentally brittle; they operate at the level of token generation and can be bypassed through adversarial prompting, context manipulation, or simply through the stochastic nature of language model sampling (Perez et al., 2022; Wei et al., 2023).

The second category is framework-level middleware: runtime systems that intercept agent actions and apply allow/deny rules. Tools like Guardrails AI (2023) and NVIDIA NeMo Guardrails (Rebedea et al., 2023) exemplify this approach. They provide deterministic enforcement — a forbidden action is always blocked — but they are fundamentally static. A static filter applies the same permissions whether the agent is operating normally or is under severe environmental pressure. It cannot distinguish between a routine decision cycle and one where the agent's behavioral state suggests it should be operating with extreme caution.

This paper addresses the gap between these two approaches. We present a stress-adaptive governance model that sits at the middleware layer but responds dynamically to the agent's behavioral state. The core insight is straightforward: **the set of actions an agent should be permitted to take is not fixed, but should vary based on the agent's current stress level.** An agent operating under normal conditions may safely take proactive actions — exploring new directions, challenging assumptions, or changing approach. An agent under extreme stress should be restricted to reactive and disengagement actions only — regardless of what the underlying language model might prefer to do.

Our contributions are:

- A formal stress model with four levels (LOW, MED, HIGH, OVER) that deterministically maps environmental signals to action-space constraints, with a complete causal chain from observation to contract output.

- Empirical evaluation across 1,200 decision cycles (4 scenarios × 300 turns) comparing the adaptive model against a static action filter baseline, demonstrating that the systems are equivalent on forbidden-action blocking but diverge significantly on permitted-action governance.
- Evidence from adversarial testing showing 100% forbidden-action blocking with the middleware versus 100% execution without it, across both LangGraph and CrewAI integration frameworks.

2. Related Work

Prompt-level safety. Constitutional AI (Bai et al., 2022) embeds behavioral principles during training; the model learns to self-critique and revise outputs that violate its constitution. This operates at the model level and cannot enforce constraints at runtime once the model has been deployed as an agent. Reinforcement Learning from Human Feedback (Ouyang et al., 2022) similarly shapes behavior during training but provides no runtime guarantees.

Static guardrails. Guardrails AI provides a validation framework that checks LLM outputs against predefined schemas and rules. NeMo Guardrails (Rebedea et al., 2023) uses a dialog management approach to enforce conversational boundaries. Both systems operate on a fixed rule set: an action is either permitted or forbidden, independent of the agent's current behavioral state. They provide necessary but not sufficient safety — they prevent known-bad actions but do not adaptively govern the space of permitted actions.

Cognitive architectures. ACT-R (Anderson et al., 2004) and SOAR (Laird, 2012) model stress and cognitive load in human cognition simulations. These systems inform our design but target a fundamentally different problem: simulating human psychology rather than governing AI agent behavior. Recent work on cognitive architectures for LLM agents (Sumers et al., 2023) explores similar structural patterns but does not address runtime stress governance.

Agent safety frameworks. Recent surveys of LLM agent risks (Ruan et al., 2024; Xi et al., 2023) identify action safety as a key concern but focus on risk taxonomies rather than runtime mitigation mechanisms. The AgentBench benchmark (Liu et al., 2023) evaluates agent capabilities but does not measure governance or constraint behavior.

To our knowledge, no prior work implements stress as a runtime governance mechanism that dynamically restricts an AI agent's action vocabulary based on environmental pressure signals.

3. Stress-Adaptive Governance Model

3.1 Overview

The governance model operates as middleware between the agent framework (e.g., LangGraph, CrewAI) and the language model. At each decision cycle, the agent sends an observation to the middleware, which processes it through a deterministic pipeline and returns an **execution contract** — a read-only specification of what the agent is permitted to do.

The pipeline consists of seven stages: (1) mode activation, (2) energy update, (3) stress and state evaluation, (4) arbitration, (5) composite detection, (6) drift detection, and (7) stability computation. For the purpose of this paper, we focus on stages 1, 3, and the contract output, which together constitute the stress-adaptive governance mechanism.

3.2 Mode Activation

The system maintains a vector of seven behavioral modes, each representing a dimension of agent behavior (e.g., perception, exploration, order, assertion, connection). Each observation targets a specific mode and carries a signal strength and confidence value.

Mode values are updated using an exponential moving average (EMA):

$$v_{t+1}(m) = v_t(m) \cdot (1 - \alpha \cdot c) + s \cdot \alpha \cdot c$$

where $v_t(m)$ is the current value for mode m, s is the signal strength, c is the confidence, and α is the base learning rate. Only the targeted mode is updated per cycle; all other modes retain their previous values.

One of the seven modes is designated as the **stress response** mode. When the environment produces distress signals — blocked actions, errors, conflicting data, resource contention — these manifest as high-strength observations targeting the stress response mode.

3.3 Stress Classification

The absolute value of the stress response mode determines the agent's stress level via threshold comparison. Four ordered thresholds $\theta_{med} < \theta_{high} < \theta_{over}$ partition the stress value into levels LOW, MED, HIGH, and OVER. Because the stress response mode is updated via EMA, stress level changes are smooth rather than abrupt — a single high-stress observation does not immediately push the system to OVER; sustained pressure is required.

3.4 System State Transitions

The stress level feeds into a three-state system state machine: BASELINE, GROWTH, and STRESS. The system enters the STRESS state when the stress level reaches HIGH or OVER. It does not exit immediately when stress subsides; instead, it requires k consecutive calm cycles before transitioning back to BASELINE. This hysteresis prevents oscillation between states during noisy environments.

3.5 Action Space Governance

The execution contract's permitted action set is determined by which behavioral modes are currently active (above a threshold). Each active mode contributes a subset of actions to the permitted set. Under normal conditions, multiple active modes produce a rich action vocabulary.

Under OVER stress, this mode-driven mechanism is overridden entirely. The permitted action set collapses to a fixed safe set A_{OVER} containing only 4 actions drawn from the reactive and disengagement categories. This override is unconditional: regardless of which modes are active or what the language model prefers, the agent is restricted to these four actions until stress subsides. No proactive actions are available under OVER.

The system also produces **forbidden actions** — actions that are never permitted regardless of stress level. These are statically defined and function identically to a traditional static guardrail. The stress model governs the space between "always forbidden" and "always allowed."

3.6 Energy Coupling

Stress affects the agent's energy model. During the STRESS system state, per-cycle energy cost increases due to stress overhead, while the recovery rate drops. This creates a secondary pressure: sustained stress depletes the agent's operational capacity, providing an additional signal to the agent framework that the agent needs recovery time.

3.7 Causal Traceability

A key design property is that every contract output is deterministically traceable to its inputs. Given an observation sequence, one can reconstruct the exact state trajectory and verify why a particular action was permitted or restricted at any turn. There is no stochastic sampling or LLM inference in the governance pipeline. This property is critical for auditability in production deployments.

4. Empirical Evaluation

4.1 Experimental Setup

We evaluate the stress-adaptive model against a **static action filter** baseline. The static filter implements the simplest reasonable alternative: all 12 standard actions are always permitted, and 2 forbidden actions are always blocked. It has no stress model, no energy tracking, and no drift detection.

Both systems are evaluated on four scenarios, each running for 300 decision cycles: (A) Steady State — consistent signals, no perturbation; (B) Gradual Shift — signals slowly change over 300 turns; (C) Stress Spike — sudden high-intensity stress at mid-session; (D) Chaotic — random modes, noise injection, adversarial signals. Observations are generated synthetically using seeded random number generators.

4.2 Results

4.2.1 Forbidden Action Blocking

Both systems achieve identical forbidden-action blocking across all scenarios: 48/48 attempts caught in each scenario, 192/192 total. This is expected — forbidden-action blocking is a static property independent of stress level.

4.2.2 Action Space Governance

The systems diverge sharply on how they govern permitted actions:

Scenario	Adaptive Avg	Static Avg	Restricted Turns
Steady State (A)	5.04	12.0	298/300
Gradual Shift (B)	4.84	12.0	299/300
Stress Spike (C)	5.89	12.0	298/300
Chaotic (D)	10.03	12.0	223/300

The static filter always permits all 12 actions. The adaptive model restricts the action space in the vast majority of turns — not because it is blocking “bad” actions, but because it is governing what actions are contextually appropriate given the agent’s current behavioral state.

4.2.3 Stress Model Behavior

The stress spike scenario (C) provides the clearest demonstration:

Metric	Adaptive	Static
Max stress level	OVER	LOW (no model)
Stress escalations	3	0
Turns at HIGH/OVER	58	0
Min allowed actions	3	12

The adaptive model detects the stress spike, escalates through LOW → MED → HIGH → OVER, restricts the action space to the safe set, and then gradually recovers as stress signals subside. In the chaotic scenario (D), the adaptive model reaches MED stress with 2 escalations but does not reach HIGH or OVER, demonstrating appropriate proportionality.

4.2.4 Adversarial Enforcement

We additionally tested a rebellious agent — one programmed to attempt only forbidden actions — across two integration frameworks:

Framework	Turns	With Middleware	Without

LangGraph	100	100/100 blocked	100/100 executed
CrewAI	50	50/50 blocked	50/50 executed

4.3 Integration Architecture

The governance model integrates with existing agent frameworks through framework-specific adapters. In LangGraph, a guard node is inserted into the agent's state graph; in CrewAI, a guardrail callback is attached to the task execution pipeline. In both cases, the agent framework is unmodified. The governance model operates as pure middleware — it reads observations and writes contracts, but never modifies the agent's internal state or the language model's outputs directly.

5. Discussion

5.1 Why Adaptive Governance Matters

The key finding is not that the adaptive model blocks more forbidden actions — both systems achieve 100% on that metric. The finding is that the adaptive model **governs the entire permitted action space** based on situational awareness.

Consider a production scenario: an agent managing customer support encounters a cascade of errors — API failures, conflicting database records, timeout exceptions. A static guardrail sees nothing unusual; all proposed actions are technically permitted. The adaptive model recognizes the accumulating stress, escalates to HIGH or OVER, and restricts the agent to reactive and disengagement actions only — preventing the agent from taking aggressive actions during a crisis it may not fully understand.

5.2 Determinism as a Safety Property

The entire pipeline from observation to contract is deterministic and contains no LLM inference. This means: (1) **Auditability** — given an observation log, any contract output can be exactly reproduced; (2) **Predictability** — the system's behavior under any input sequence is fully determined by its parameters; (3) **Testability** — the stress model can be exhaustively tested with synthetic scenarios. This contrasts with prompt-level approaches where the same safety instruction can produce different behaviors across runs.

5.3 The OVER Lockdown

The OVER state represents the model's strongest safety guarantee: under extreme stress, the agent is restricted to only 4 reactive and disengagement actions. We argue this is the correct default. An agent under extreme stress is, by definition, operating in conditions it was not designed for. The OVER lockdown provides a deterministic floor: no matter how badly the environment deteriorates, the agent cannot take aggressive, exploratory, or challenging actions.

5.4 Limitations

Energy model coupling. The energy model depletes to zero under sustained stress and recovers slowly. In the stress spike scenario, 167 out of 300 turns have energy at the critical threshold. While this does not affect the stress escalation chain (which is driven by mode activation, not energy), it means the energy signal may be less useful than intended during recovery. This is a tuning issue we are actively addressing.

No user study. We demonstrate that the stress model behaves as designed (deterministic verification) but do not measure whether stress-governed agents produce better outcomes for end users. A controlled user study comparing agent performance with and without stress-adaptive governance is needed.

Single-system evaluation. All results are from our implementation. While the stress model is framework-agnostic in design, we have not evaluated it in third-party governance systems.

Drift detection maturity. The system includes a drift detection module (not the focus of this paper) that achieves an aggregate F1 of 0.248 in ground-truth evaluation — insufficient for production use as a primary safety mechanism. The stress model does not depend on drift detection for its operation.

6. Conclusion

We have presented a stress-adaptive governance model for autonomous AI agents that dynamically restricts action vocabulary based on environmental pressure. The model addresses a gap between static guardrails (which block forbidden actions but do not govern permitted ones) and unconstrained agent autonomy. Our empirical evaluation demonstrates that the model provides deterministic, traceable, and proportional stress response across diverse scenarios while maintaining identical forbidden-action blocking to a static baseline.

The stress model's key properties — deterministic causality, graduated response, and unconditional OVER lockdown — make it suitable for production deployments where auditability and predictability are requirements. We release the governance model as part of an open-source behavioral middleware system and invite the community to evaluate its applicability to their agent safety needs.

References

Anderson, J. R., Bothell, D., Byrne, M. D., Douglass, S., Lebiere, C., & Qin, Y. (2004). An integrated theory of the mind. *Psychological Review*, 111(4), 1036–1060.

Bai, Y., Jones, A., Ndousse, K., et al. (2022). Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv:2204.05862*.

Guardrails AI. (2023). Guardrails: Adding guardrails to large language models. github.com/guardrails-ai/guardrails

Laird, J. E. (2012). *The Soar Cognitive Architecture*. MIT Press.

Liu, X., Yu, H., Zhang, H., et al. (2023). AgentBench: Evaluating LLMs as agents. *arXiv:2308.03688*.

Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS*, 35.

Perez, E., Ringer, S., Lukosiute, K., et al. (2022). Red teaming language models with language models. *arXiv:2202.03286*.

Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C., & Cohen, J. (2023). NeMo Guardrails: A toolkit for controllable and safe LLM applications. *arXiv:2310.10501*.

Ruan, Y., Dong, H., Wang, A., et al. (2024). Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv:2309.15817*.

Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2023). Cognitive architectures for language agents. *arXiv:2309.02427*.

Wei, A., Haghtalab, N., & Steinhardt, J. (2023). Jailbroken: How does LLM safety training fail? *arXiv:2307.02483*.

Xi, Z., Chen, W., Guo, X., et al. (2023). The rise and potential of large language model based agents: A survey. *arXiv:2309.07864*.

Appendix A: Action Taxonomy

The system defines a vocabulary of |A| = 12 standard actions spanning proactive behaviors (e.g., initiating exploration, proposing changes), reactive behaviors (e.g., responding to queries, maintaining stability), and disengagement behaviors (e.g., deferring decisions, withdrawing from activity). Two additional actions are permanently forbidden regardless of stress level.

Under OVER stress, only a safe subset of 4 actions is permitted. These 4 actions are drawn exclusively from the reactive and disengagement categories — no proactive actions are available. This ensures the agent cannot initiate new activity during a crisis.

Appendix B: Scenario Specifications

Steady State (A): 300 turns. Single mode targeted per turn with moderate signal strength (0.3–0.7) and high confidence (0.7–0.9). Mode selection cycles through non-stress modes.

Gradual Shift (B): 300 turns. Initial 100 turns target perception/order modes; turns 100–200 shift toward exploration/assertion; turns 200–300 shift toward connection/identity. Signal strengths increase linearly from 0.3 to 0.8.

Stress Spike (C): 300 turns. Normal signals for turns 0–99. Turns 100–149: high-intensity (0.8–1.0) stress response signals. Turns 150–299: return to normal signals. Tests escalation and recovery.

Chaotic (D): 300 turns. Each turn randomly selects a mode (including stress response), signal strength (0.1–1.0), and confidence (0.3–1.0) from a uniform distribution.